Pretraining Cardinality Estimators on Less Datasets: Finding Special Databases when Training

Boyang Fang
2110672@stu.neu.edu.cn
Northeast University
Shenyang, China

Tiezheng Nie
nietiezheng@cse.neu.edu.cn
Northeast University
Shenyang, China

Quanqing Xu
oceanbase
Hangzhou, China

Derong Shen
Northeast University
Shenyang, China
shenderong@cse.neu.edu.cn

Yue Kou
Northeast University
Shenyang, China
kouyue@cse.neu.edu.cn

Ge Yu
Northeast University
Shenyang, China
yuge@cse.neu.edu.cn

Abstract

In cardinality estimation research, zero-shot models have become increasingly prevalent. Recent pretrained cardinality estimation methods, trained on diverse datasets sourced from multiple databases, have achieved high accuracy. By adapting the Group Distributionally Robust Optimization (Group DRO) algorithm to the training process, we find that some specific databases contribute more significantly to model performance than others. To investigate whether the existence of such databases is independent of any single model, we introduce a new pretrained cardinality estimation model and an algorithm to rank the order of these training databases to explain why they are more significant. Based on these observation, we conduct extensive experiments to delve deeper into pretrained cardinality estimators. Our experimental results indicate that only a subset of the databases contributed meaningfully when training the zero-shot model, revealing significant redundancy in the current composition of training databases. We introduce some simplified training datasets that have been reduced to a fraction of the size of existing pretraining datasets. Sufficient experimental results demonstrate that the pretrained cardinality estimator based on these simplified datasets can still achieve comparable performance to existing models in zero-shot setups.

Keywords

AI4DB, cardinality estimation, query optimization

ACM Reference Format:

Boyang Fang, Tiezheng Nie, Quanqing Xu, Derong Shen, Yue Kou, and Ge Yu. 2018. Pretraining Cardinality Estimators on Less Datasets: Finding Special Databases when Training. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX).* ACM, New York, NY, USA, 12 pages. https://doi.org/XXXXXXX.XXXXXXX

Conference acronym 'XX, Woodstock, NY

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

When a database system processes a given SQL query, how can it obtain accurate query results as quickly as possible? The query optimizer in a database is specifically designed to address this problem. The article How Good Are Query Optimizers, Really?[3], which introduced the JOB benchmark, argues that the cardinality estimator, cost model, and plan enumeration are the core components of an optimizer that generate optimal query plans, with the cardinality estimator having a greater impact on performance. More accurate cardinality estimation has long been a hot topic in database research, with numerous articles over the decades proposing methods to improve its precision. In recent years, as machine learning models have demonstrated powerful capabilities, database researchers have also attempted to take advantage of them to solve the cardinality estimation problem.

Research on cardinality estimation, particularly with the application of machine learning methods, can be broadly categorized into traditional approaches and learning-based approaches. The latter, depending on the learning objective, are primarily divided into two directions: query-driven and data-driven[7, 13, 15, 18, 25, 27, 28, 30, 32, 33, 35]. Most learning-based methods construct models based on a single database instance or a workload tailored to that specific instance. Under stable data distributions or static workloads, many of these methods have achieved state-of-the-art (SOTA) performance. However, models trained under such conditions lack transferability across different databases, a limitation that restricts the practical deployment of learning-based methods in real-world database systems. To address this constraint, several zero-shot models[6, 12, 18, 24] tailored for database optimization problems have been proposed, demonstrating remarkable effectiveness. Inspired by zero-shot learning in natural language processing (NLP), these models are trained on a diverse and heterogeneous collection of database instances and their corresponding workloads. Once pretrained, they can be directly applied to entirely unseen databases without further fine-tuning, achieving performance comparable to other learning-based methods. Furthermore, their performance can be enhanced on the target database through few-shot adaptation with minimal additional data.

However, existing work on pretrained cardinality estimators[6, 18, 32] has not yet sufficiently investigated or tested these models in depth. Key factors such as the composition of the pretraining

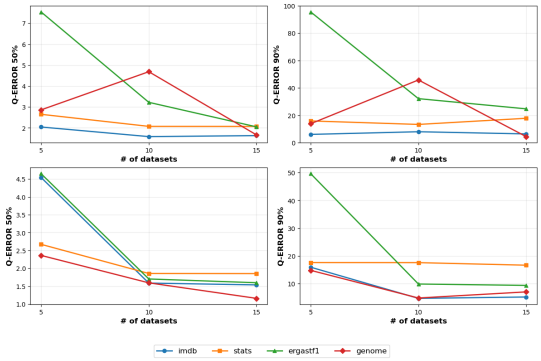

Figure 1: Evaluation of PRICE's performance with varied quantities of datasets. Upper row: Select up to 15 databases following an ascending order of table count. Bottom row: Select up to 15 databases following a descending order of table count.

dataset, the type and architecture of the base model, and their impact on the final query optimizers remain unclear. For example, the composition of their training datasets exhibits a high degree of overlap in terms of the source databases, and a common ablation study in the existing literature[6, 12, 18, 32] shows that model performance generally improves as the number of databases used during training increases. However, as illustrated in Figure 1, when we select training datasets based on the number of tables per database, the resulting performance curves differ significantly in shape.

In the upper section of the figure, we incrementally selected up to 15 databases in ascending order of table count. When the number of databases reached 10, each contained no more than 6 tables. At this stage, model performance improved only marginally and even decreased on some test databases. In contrast, the two lower subfigures show performance curves when databases were selected in descending order of table count. By the time 10 databases were included, each contained at least 8 tables. The resulting curves align more closely with the trends typically reported in ablation studies. This phenomenon suggests that the structure of the databases used in training, which is not merely their quantity, plays a critical role in the performance of pretrained cardinality estimators.

In summary, the main contributions of this are as follows:

1) During the training of the PRICE pretrained model[32], we built upon the DoReMi algorithm to quantify the contribution of each database to the model's performance. We introduced an excess loss-based scoring method to assess the importance of each database, identifying which subsets of the training data contribute most significantly to final performance. Based on this analysis, we propose several simplified training sets.

2) To validate the effectiveness, generalization capability, and stability of these simplified training sets, we developed a new pretrained model named Simulated PRICE.

3) Through extensive experiments, we demonstrate that both the standard PRICE model trained on simplified training datasets and the Simulated PRICE model achieve comparable performance to the full-size training dataset baseline while significantly reducing training costs. Notably, in some test databases, the simplified versions even yield improved performance.

The code used in this article is open source at https://github.com/spiceandwolf/PRICE.git.

2 Preliminaries

2.1 Problem Formalization

Let a database instance be defined by a set of tables $T = \{T_1, T_2, ..., T_n\}$, where each table T_i comprises n attributes $T_i = \{A_1, A_2, ..., A_m\}$. Typically, we constrain the data type of each attribute to be either numerical or continuous. The domain of attribute A_i can be represented as $Dom(A_i) = \{c_1, c_2, ..., c_p\}$ for discrete values or as $Dom(A_i) = [min_i, max_i]$ for continuous ranges. In this article, we primarily focus on select-project-join (SPJ) queries with conjunctive predicates, while excluding LIKE-type queries on strings and not considering Approximate Query Processing (AQP). The queries we primarily discuss adhere to the following form:

$$SELECT\ COUNT(*)\ FROM\ T_q\ WHERE\ J\ AND\ F. \quad (1)$$

These queries typically consist of (1) A subset of tables $T_q \subseteq T$ involved in the query, (2) Join conditions J (exclusively primary-key/foreign-key equi-joins in the form $T_i.A_b = T_j.A_c$, (3) Filter predicates F (triples $(T_i.A_b, o_p, v_i)$ where $T_i.A_b$ is a column attribute, $o_p \in \{=, >, <, \geq, \leq\}$ and $v_i \in Dom(A_j)$). Each such SQL query q and its true cardinality $card(q)$ form a tuple $x_i = \langle q, card(q)\rangle$. The goal of cardinality estimation is to efficiently compute an estimated cardinality $\overrightarrow{\text{card}}(q)$ without physically executing q, while maximizing precision.

For a given database instance, multiple tuples x_t are combined to form a set $\mathcal{X}_i = \{x_1, x_2, ..., x_t\}$, referred to as a workload. After training a learning-based method, a workload is selected as a test set, where the model estimates cardinality $\overrightarrow{\text{card}}(q)$ for all queries q in the test workload. The performance of the model is then evaluated using the q-error metric, calculated between the true cardinality $\text{card}(q)$ and the estimated cardinality $\overrightarrow{\text{card}}(q)$.

For query-driven approaches, the training process additionally requires a separate training workload that has no overlap with the test set. Current pretrained cardinality estimation models utilize a training set composed of workloads aggregated from multiple database instances. Thus, the pretraining dataset is formally represented as $\mathcal{X} = \{\mathcal{X}_1, \mathcal{X}_2, ..., \mathcal{X}_i\}$

2.2 Pretrained Cardinality Estimators Revisited

The remarkable success of pretrained models in NLP, CV, and other domains, where a single model trained once can solve diverse downstream tasks, has made them highly attractive for query optimization. In the context of query optimization, existing pretrained models vary in design: some works [24] pretrain on a single database to produce a model capable of handling multiple query optimization tasks within that same database; others [6, 12] train across multiple databases to create a model specialized for a single specific optimization task. The pretrained cardinality estimators we discuss in the following fall into the latter category.

Unlike query-based or data-driven models, current pretrained cardinality estimators no longer rely solely on learning from query-specific information or raw data statistics. Instead, they are built upon summarized representations of the underlying data. The Iris model[18], structured using an encoder and a decoder, was pretrained on 18 single-table datasets. As demonstrated in the article, Iris serves as a lightweight and rapid estimation alternative when traditional summarization methods (e.g., Sparse and histograms) are infeasible. Even in zero-shot scenarios, Iris achieves high accuracy with a small storage overhead. The CardBench study[6] introduced two pretrained models (GNN-based and Graph Transformer-based) using 19 multitable datasets. Although capable of handling single-table queries and two-table join queries, their zero-shot accuracy proved limited, though fine-tuning yielded significant improvements. PRICE[32] leverages a multi-head self-attention mechanism between the attribute embedding vectors of a dataset and a join condition embedding vectors, and another multi-head self-attention mechanism between the data distribution embeddings and the query information embeddings. This architecture enables cardinality estimation for queries on any unseen dataset. Pretrained on 26 multi-table datasets, PRICE maintains high accuracy even in zero-shot scenarios. In summary, PRICE exhibits superior generalization and estimation performance, fulfilling the core requirements for pretrained models. Therefore, in the subsequent sections, we adopt the PRICE model as our baseline and focus primarily on multi-table join scenarios.

Existing work on pretrained cardinality estimators typically addresses two key challenges:(a) Identifying transferable features that capture both the structural characteristics of datasets and the statistical properties of SQL queries executed on them; (b) Constructing training datasets, where current approaches predominantly combine heterogeneous domain-specific datasets with their corresponding workloads.

2.2.1 transferable features identification. Encoding transferable features is a necessary step in applying pretrained cardinality estimators to query optimization. Like query-driven learning models, pretrained models require executing a certain number of SQL workloads and collecting them as training data before training. The main difference is that pretrained models need to generalize to unseen datasets, where the queries are OOD (out-of-distribution). In contrast, query-driven methods rely solely on features derived solely from training SQL queries, typically information such as tables, joins, and predicates, making them constrained to a specific database instance and the corresponding training workloads. Consequently, query-driven methods' performance significantly degrades when faced with OOD SQL queries. For example, in the STATS dataset, the column **commentcount** might be one-hot encoded as (0, 0, 1, 0) during the encoding phase. However, if the model is later applied to the IMDB dataset, the same one-hot encoding (0, 0, 1, 0) might represent a completely different column, such as **production_year**. These columns have entirely different data distributions, including data types, making the encoded features nontransferable. As a result, the model will likely not perform effectively. On the other hand, while data-driven learning models do not rely on encoding information such as tables, joins, and predicates, they model the underlying distribution of each dataset. This results in data-driven learning models being nontransferable across different datasets.

Pretrained models require us to represent query information in a unified format across different databases using an appropriate transferable feature encoding method, while also demanding sufficient expressive power to achieve accurate estimation. Existing approaches to this problem can be broadly divided into two categories: methods like PRICE, which use the combination of low-level attribute information with corresponding coefficients; and others like CardBench, which rely on graph-structured data. The features they selected include various statistical information for each attribute, aiming to learn the data distribution. However, PRICE's strategy of dividing the cardinality estimation process into multiple stages grants it greater flexibility than CardBench when handling joins involving varying numbers of tables.

2.2.2 training dataset construction. Existing pretrained cardinality estimators[6, 18, 32] require abundant training data, typically built upon multiple datasets of different types, including both synthetic datasets and those derived from real-world instances. Beyond the datasets themselves, the training data also encompasses queries executed on each dataset, runtime statistics collected during query execution, the corresponding true cardinalities, and query plans. In other fields, such as NLP, research on pretrained models has been extensively explored and there are studies demonstrating that increasing the diversity of training datasets enhances the performance of pretrained models in zero-shot situations. Training data is typically created by sampling and mixing text-based examples from numerous domains.

However, for the cardinality estimation task, after selecting a dataset, it is still necessary to generate a sufficient number of queries to form the training workload. The process of generating a single query usually begins with randomly selecting a subgraph from the database's join schema graph. Based on this join subgraph, the corresponding columns and join information are collected from the involved tables. An integer n, representing the total number of predicates, is then sampled from a uniform distribution ranging from 1 to the total number of columns. For each of the n steps, an attribute not already used in the join conditions is randomly selected from within the subgraph, and a corresponding filter predicate is generated. Depending on whether the attribute is numerical or categorical, the filter predicate is designated as either a range query or an equality query. The generated queries along with their true cardinalities constitute the training data. Each distinct type of join schema graph corresponds to a different category of training set.

For ML models, high-quality training data should be **Learnable**, **Worth Learning**, and **Not Yet Learnt**[19], and the more high-quality training data a domain contains, the greater its contribution to model performance. Current pretrained cardinality estimators construct their training datasets by mixing workloads collected from multiple databases. From the perspective of their role in the training process, these different datasets are analogous to the distinct domains used in language model training process. Some existing studies[8, 11, 20, 29] have demonstrated that the composition ratio of different domains among the training datasets can significantly impact model performance. However, current pretrained cardinality estimators simply assign equal weight to each

training dataset $\mathcal{X}_i$, meaning all training datasets contain the same number of tuples $|\mathcal{X}_i|$.

3 Quantifying Domain-Specific Contributions in Training Data

In this section, we present an algorithm to evaluate the contribution of each domain in the training dataset $\mathcal{X}$ to the performance of a pretrained cardinality estimator. Based on this, in the following sections, we optimize the composition of the training dataset $\mathcal{X}$ by reweighting different domains accordingly.

Assume the training dataset of a pretrained cardinality estimator, denoted as $\mathcal{X}_i = \{x_1, x_2, ..., x_t\}$, is mixed from k training datasets. The weights corresponding to each dataset form a probability simplex $\alpha \in \Delta^k \subset \mathbb{R}^k$. The distribution of the training dataset $\mathcal{X}$ can be expressed as $P_\alpha = \sum_{i=1}^{k} \alpha_i \cdot UNIF(X_i)$, with each dataset $\mathcal{X}_i$ containing tuples uniformly sampled from the workload data distribution of its corresponding domain. For a pretrained cardinality estimator θ, the standard loss function $L(\theta)$ always employs Mean Squared Error (MSE) to quantify the discrepancy between the true cardinality values and the model's predictions. The loss function for each training dataset $\mathcal{X}_i$ is defined as $l_i(\theta) = \frac{1}{|\mathcal{X}_i|} \sum_{j=1}^{|\mathcal{X}_i|} (log(card(q_j)) - log(\overrightarrow{\text{card}(q_j)}))^2$, where $|\mathcal{X}_i|$ is the number of SQL queries in the training dataset $\mathcal{X}_i$. After incorporating the weights for each training dataset, the original loss function of the pretrained cardinality estimator can become a domain-specific loss: $l_\theta = \sum_{i=1}^{k} \alpha_i \cdot l_i(\theta)$, where $\alpha_i = \frac{|\mathcal{X}_i|}{|\mathcal{X}|}$.

The pretrained cardinality estimator is expected to be directly applicable to any unseen new database instance, just like traditional methods. Therefore, when quantifying the contribution of each domain in the training dataset to the model, we must avoid dependence on any specific database instance. Referring to approaches like DoReMi[29] and DRO-LM[20], we determine the better weights for each domain by minimizing the worst-case training loss of a proxy model θ under the distribution P_α, in order to enhance the model's generalization capability without requiring any prior knowledge of the target domain.

In this scenario, the objective of solving the problem becomes:

$$L(\theta, \alpha) := \underset{\theta}{\text{argmin}}\{R(\theta) := \max_{\alpha \in \Delta^k} \sum_{i=1}^{k} \alpha_i \cdot \left[\sum_{x \in X_i} max(l_\theta(x) - l_{fin}(x), 0) \right]\} \tag{2}$$

In (2), both l_θ and l_{fin} are loss functions of the pretrained cardinality estimator. The former is the loss function of the proxy model θ, while the latter serves as a reference model to help evaluate the training progress of the proxy model in each dataset. Since pretrained cardinality estimators are often time and space efficient, we can directly use the optimal models provided by existing research as reference models.

More formally, this approach is based on distributionally robust optimization (DRO). Under this framework, given in a training step t and its corresponding input data batch b, the model training progress consists of two steps:

- Step 1. Compute the performance gap between the current proxy model θ and the reference model, then update the corresponding weights for each training dataset. In this step, we first compute the gap between the two models' MSE losses for each domain separately. Our goal is to quantify the contribution of each domain in the training dataset to the performance of the reference model, which requires that the proxy model θ performs no worse than the reference model in any domain. We clip the loss gaps that are less than zero because, the proxy model θ has already **Learnt** for such samples, meaning its performance on these samples is no worse than that of the reference model. When a sample is **Learnt**, it means the model can achieve sufficient fitting without requiring more data than what has been used before the next step $t + 1$. Therefore, domains with higher losses contain more data that is **Learnable**, **Worth Learning**, and **Not Yet Learnt**. Simultaneously, we record the information of all samples whose excess loss remains non-zero at the current epoch. Since the pretrained cardinality estimator is inherently a regression model, we update the domain weights here using the formula from the Group DRO optimizer[22], based on the mean training loss in batches of each domain:

$$\alpha_{i,t} \leftarrow \alpha_{i,t-1} \cdot e^{l_i(\theta)} \tag{3}$$

- Step 2. We compute the total loss $L(\theta, \alpha_t)$ using the probability simplex α_t that represents the new domain weights and each dataset's corresponding training loss $l_i(\theta)$ at the current training step t. Then update the parameters of the proxy model θ via stochastic gradient descent using a standard optimizer such as Adam[14].
- Step 3: We compare the differences in excess loss for samples from each domain during training. The excess losses of all samples are sorted per epoch, with the goal of obtaining, for each domain, two metrics: topw (the count of its samples among the topw samples with the highest losses) and bottomw (the count among the bottomw samples with the lowest losses). We regard the former as the hardest examples and the latter as the easier examples. The difference between topw and bottomw is used to quantify the overall distribution of excess loss because a larger difference indicates a greater concentration of high excess loss within the corresponding database. Furthermore, since this method only requires identifying the topw and bottomw samples, it avoids the need to sort the entire dataset, thereby reducing computational complexity.

The idea of statistically analyzing sample-wise excess loss in Step 3 is inspired by prior work in data pruning[21, 23]. Pseudocode for this procedure is provided in Algorithm 1. In addition to using excess loss to measure the performance gap between the current model and the target model, this step allows us to track how well the training samples from each domain are learned after every epoch. In the original DoReMi method, if samples from certain domains exhibit high loss variance, even if the number of truly learnable samples in those domains is small, it can still lead to inflated values of α. Therefore, it is necessary to more comprehensively account for the overall influence of each domain's training data on model performance.

After training is completed, the full dataset can be simplified by subsampling according to the final domain weights α. This

Algorithm 1 Ranking Based on Excess Loss Differences

Require: excess losses for each domain per epoch $l_{k,t}$
Hyperparameters: count window w, number of training epochs T, number of domains K, smoothing parameter c (e.g., c = 0.5 in our implementation)
init domain rank $D_{k,t}$
for t from 0 to T **do**
Obtain score list $S_{k,t}$ and number of learnt examples l[k,t] for each domain k $\in \{1, 2, ..., K\}$:
$S_{k,t} \leftarrow \max\left\{l_{k,t}, 0\right\}$
$l[k,t] = |l_{k,t}| - |S_{k,t}|$
Obtain the number of hardest examples h[k,t] and the number of easier examples e[k,t] in topw(S_t) and bottomw(S_t) for each domain k $\in \{1, ..., K\}$, $S_t = \left\{S_{1,t}, S_{2,t}, ..., S_{K,t}\right\}$:
$h[k,t] = |topw_k(S_t)|$
$e[k,t] = |bottomw_k(S_t)|$
if t = 0 **then**
$D_{k,t} = h[k,t] - e[k,t]$
else
$D_{k,t} = (1-c) * (h[k,t] - e[k,t]) + c * D_{k,t-1}$
end if
end for
return domain rank $D_{k,t}$

process can be further refined by incorporating the sample rankings obtained in Step 3 to optimize the final subset selection.

4 More Pretrained Cardinality Estimator

To demonstrate the effectiveness and generalizability of our optimized training dataset, it is necessary to evaluate it on models beyond PRICE. However, at present, no other zero-shot pretrained models directly applicable to multi-table join cardinality estimation are available: CardBench focuses primarily on binary joins, while [12] targets cost estimation. Therefore, in this section, we draw inspiration from these works to propose an alternative zero-shot pretrained model distinct from PRICE. As noted in [12], zero-shot models must encode query information using transferable features. In our approach, we replace one-hot encodings of tables with histograms of the involved columns in the query, and represent joins between two tables using scaling factors. We further employ attention mechanisms to capture relationships among these feature embeddings. This results in a novel zero-shot model, whose architecture is illustrated in Figure2.

To ensure rigorous validation, we guaranteed that the encoded features introduced no additional information compared to the PRICE model, and similarly utilized a virtual node vector to aggregate all information. Unlike PRICE, however, our approach first computes the embedding of the entire join subgraph using all related column histograms, scaling factor histograms, and the sizes of tables in the query predications. This embedding is then combined with the embeddings of the query predicates and the corresponding table statistics. Finally, we still train a regression model with an MLP using the virtual vector.

5 Experiment Studies

In previous sections, we introduced a method for estimating the contribution of training datasets from different databases to model performance and proposed a new pretrained cardinality estimator to enhance generalizability. In this section, we evaluate the performance of this new pretrained cardinality estimator alongside PRICE, comparing them with the original PRICE model and other existing state-of-the-art (SOTA) techniques for multi-table cardinality estimation. Our experiments are designed to address the following questions:

(1) One of our key ideas for simplifying the training set is to sample data proportionally to the contribution of each database to model performance. How will models trained on such a simplified dataset perform when directly applied to unseen datasets?

(2) Inspired by the effectiveness of models trained on simplified datasets: if certain databases in the original training set are redundant, how would models trained only on a few specific datasets perform on unseen data? Can such a curated training set generalize to other pretrained cardinality estimators?

(3) What are the characteristics of the query distribution in such simplified training sets? What advantages do they offer compared to the original training set?

5.1 Experiment Setup

5.1.1 Dataset. In the field of query optimization, existing pretrained zero-shot methods[6, 12, 32] utilize datasets derived from a total of 30 databases, which exhibit substantial overlap and consist exclusively of multi-table join databases. To ensure a fair comparison, we adopt the same 26 databases as PRICE[32] for model training, reserving the remaining 4 databases for performance evaluation. The unseen test set comprises four real-world databases: IMDB[3], STATS[4], ErgastF1[1], and Visual Genome[2]. The IMDB dataset is a classic benchmark widely used in the database field. The STATS dataset is an anonymized dump of user-contributed content on the STATS Stack Exchange network. ErgastF1 is a dataset about Formula 1 race information. Visual Genome is a dataset, a knowledge base, and an ongoing effort to connect structured image concepts to language.

5.1.2 Workloads. Among these four test datasets, the first two datasets correspond to the workloads JOB-light[16] and STATS-CEB[10], containing 70 queries and 146 queries for testing, respectively. The latter two datasets use the same test workloads provided in the PRICE article, consisting of 148 and 186 queries for each.

Furthermore, we chose 7 more datasets from the original 30 datasets for additional testing. Table 1 provides the statistics of the datasets used in these experiments.

5.1.3 Evaluation Metrics. We chose Q-error, P-error and End-to-End(E2E) Time as the primary metrics to evaluate model performance.

Q-error. Cardinality estimation problems often use q-error as an accuracy metric, which is calculated according to the formula:

$$\text{Q-error} = \max\left(\frac{\overrightarrow{\text{card}(Q)}}{\text{card}(Q)}, \frac{\text{card}(Q)}{\overrightarrow{\text{card}(Q)}}\right) \tag{4}$$

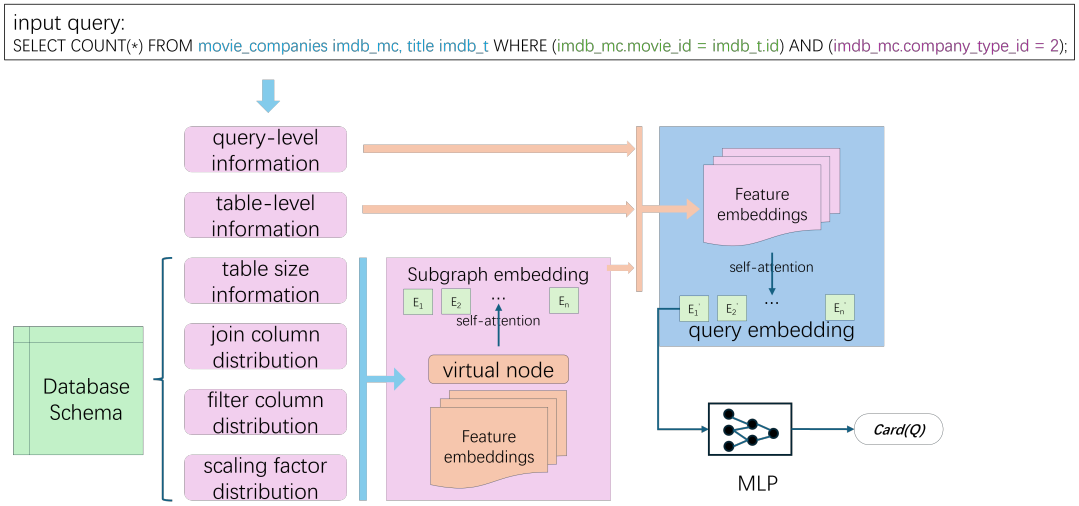

Figure 2: Model architecture of Simulated PRICE.

Table 1: Statistics of Datasets

Dataset Name	Join Forms	Average Pairwise Correlation	Total Number of Cols
Accidents	Chain	0.33	43
Carcinogenesis	Star	0.69	24
Consumer	Chain	0.40	25
Hockey	Mixed	0.33	300
SSB	Star	0.13	58
TalkingData	Mixed	0.65	25
Baseball	Mixed	0.38	359

the range of a Q-error is between $[1, \infty)$. However, this metric can not distinguish whether the relationship between the estimated cardinality and the true cardinality is an overestimate or an underestimate. When generating an execution plan, the impact of an overestimate or underestimate of the cardinality on the final End-to-End time is agnostic.

P-error. Although the Q-error metric is often used to measure the accuracy of the estimated cardinality, it cannot serve as a good indicator for query execution performance. This is because both overestimates and underestimates of cardinality can yield the same Q-error but result in completely different query plans, leading to significantly different query times. Although the best way to evaluate the quality of cardinality estimation is by directly comparing the actual execution times on the same benchmark, this requires a substantial time cost. Therefore, the P-error metric was proposed in [10].

To generate an execution plan for a query q, it is first necessary to estimate the cardinality of the corresponding subqueries. The true cardinality set and the estimated cardinality set of the subqueries are denoted as C_T and C_E, respectively. When calculating the P-error, PostgreSQL[5] is used to output the corresponding query plans $P(C_T)$ and $P(C_E)$ based on C_T and C_E. The cost model of the PostgreSQL query optimizer takes the query plan P and the required cardinality set C as inputs, and its output, the estimated execution cost, is denoted as $PPC(P(C), C)$. Thus, p-error is defined as:

$$\text{P-error} = \frac{PPC(P(C_E), C_T)}{PPC(P(C_T), C_T)} \tag{5}$$

This metric can measure the differences at the query plan level between the estimated cardinality and the true cardinality, therefore it can serve as a proxy for the End-to-End time metric to a certain extent.

End-to-End Time. End-to-end Time refers to the total time from when an SQL query enters the database system until the final results are returned, serving as the gold standard metric for evaluating database performance. Compared to Q-error and P-error, which are purely mathematical metrics, End-to-End time is influenced by multiple factors beyond estimation quality including hardware conditions. To mitigate measurement noise, we use the average End-to-End time over five times repeated executions of the same query batch. To measure the End-to-End time of each learned cardinality estimator, we execute queries in PostgreSQL while leveraging the cardinality estimates of the relevant subqueries.

5.1.4 baselines. We selected several state-of-the-art (SOTA) learned cardinality estimators as baselines, each based on different machine learning models. We trained the pretrained cardinality estimator using the remixed training dataset and compared its performance not only with the original model but also with these baseline models.

- MSCN[15]. It is a query-driven model that utilizes a multi-set convolutional network to train a regression model based on the workload from a specific dataset.
- NeuroCard[30]. It is an improved version of Naru adapted for multi-table datasets. Naru[31], a data-driven model based on autoregressive modeling. NeuroCard primarily supports acyclic join operations.
- Postgres[5]. It serves as the represent of traditional cardinality estimators, primarily based on the 1-D histogram based

approach. As such, it is a universally adopted baseline for benchmarking performance across most models.
- PRICE[32]. It is a pretrained model based on the self-attention mechanism, trained via a query-driven supervised approach with inputs including histograms and statistical information of columns involved in the queries.
- QSPN[17]. It is a data-driven approach based on Sum-Product Networks (SPNs), which enhances the FLAT[35] model by incorporating the ability to learn query-specific information and further optimizes the method to handle multi-table join queries.

For these methods, we apply the original code implementations provided in the articles to the test datasets and primarily follow their native parameter configurations. We excluded NeuroCard from execution as it specifically supports the tree-like schema when the dataset schema contains cycles. For fairness, we also apply 5×10^4 queries on each dataset to train MSCN.

5.1.5 Training Environment. The Linux server is equipped with an Intel Xeon Gold 5218R CPU, 128GB of memory, and a NVIDIA A6000 GPU.

5.1.6 ours. When estimating the contribution of each domain in the training set using the PRICE training process combined with the algorithm described earlier, we adhere to the original hyperparameter configuration for the proxy model as specified in the PRICE article. In subsequent experiments, since the simplified training set contains fewer samples, we appropriately adjust the learning rate during training. Once the weights α for the training dataset $\mathcal{X}$ stabilize during convergence, we perform a random uniform sampling from each original training dataset $\mathcal{X}_i$ based on the updated weight proportions α_i. Due to the space limit, the details of the weights are explained in the Appendix A. Since each domain in PRICE's training dataset contains 5×10^4 SQL queries per workload, we allow only the domain with the highest weight α_{max} to use the full workload for fairness. For other datasets $\mathcal{X}_i$ in $\mathcal{X}$, the sampling ratio was scaled proportionally by $\alpha_i^* = \frac{\alpha_i}{\alpha_{max}}$. The reweighted training dataset $\mathcal{X}_{simple}$ contains approximately 1.1×10^5 SQL queries in total, and we train the PRICE model called Simple PRICE on the reweighted training dataset.

5.2 Performance of Simplified Datasets

We compared the Q-error and P-error of Simple PRICE and other baselines across these four unseen new datasets, as shown in Table 2. In this table, both Simple PRICE and PRICE are zero-shot performance. Since the Q-error and P-error metrics are solely related to logical operations, the methods marked with $*$ use the results from [32].

We now discuss a few observations. Compared to other baselines, Simple PRICE demonstrates superior performance in most quantiles of Q-error and P-error. In particular, it shows a clear advantage compared with MSCN which presents the query-driven model below the 95th percentile across all test datasets. A lower Q-error indicates higher accuracy, while a lower P-error indicates closer similarity with the optimal execution plan. When compared to NeuroCard (representing autoregressive models) and QSPN (representing SPN-based methods), Simple PRICE exhibits greater precision in estimating results, especially in complex join schemas like STATS. Compared to PRICE, Simple PRICE achieves superior performance on two of the four unseen datasets (On IMDB: 95% Q-error: 8.48 vs. 15.45 (lower is better); 99% Q-error: 47.27 vs 70.89). Notably, Simple PRICE achieves this while using only 10% of the original training data (1.1×10^5 SQL queries vs. 1.3×10^6 SQL queries), significantly reducing training overhead.

The performance comparison between Simple PRICE and PRICE on these four unseen new datasets suggests significant room to improve in the training datasets of current pretrained cardinality estimators. By examining the probability simplex α assigned to each domain, as shown in Appendix A, we observe that their distribution is not uniform. The **Learnable**, **Worth Learning**, and **Not Yet Learnt** data are concentrated in only a few domains. In some datasets $\mathcal{X}_i$, the weights α_i are too small ($<1e-4$) to randomly sample an example given a workload having 5×10^4 SQL queries per domain. Simple PRICE's performance demonstrates that removing these extremely low-weight domains from the training dataset does not severely degrade model performance. This implies that the knowledge from these domains may already be contained in the primary contributing datasets.

Reviewing the design ideas of existing pretrained models, they all aim to represent the high-level PDF $Pr(T)$ by combining low-level attribute information. Here, the high-level PDF $Pr(T)$ refers to the result of an outer join across multiple tables in a dataset, while the low-level attribute information is the data distribution of each attribute A_i. They hope that the adaptiveness can be attained by a pretrained model, which has witnessed enough number of diversified data distributions and could learn to produce the corresponding coefficients with its internal parameters to combine low-level information into high-level joint PDFs. Consequently, the strategy of exiting pretrained models involves aggregating multiple workloads from different datasets to create a richer training dataset in order to incorporate more low-level attribute information. The condition of a database instance having a substantial number of tables and columns can also yield a wealth of low-level attribute information.

The PRICE article[32] explored how the size of the training dataset impacts the pretrained PRICE. However, their experiments did not specify which exact datasets were included when varying the number of datasets in the training data. The findings from our experiments demonstrate that model performance is dominated by a handful of databases. This implies that the information provided by the training samples in the remaining databases is likely already captured by this dominant subset.

5.3 Performance of zero shot models and those pretrained on specific databases

Our previously proposed novel zero-shot pretrained model, employing the same hyperparameter settings as PRICE and likewise tested on four unseen databases, has its results displayed in Table 2, using "Simulated PRICE" as the label. The performance of Simple PRICE indicates that there is redundancy among the 26 databases used for training. During our experiments, we identified a specific subset of these databases, consisting of five databases: 'accidents',

Table 2: Overall Estimation Accuracy of different CardEst methods.

DATASETS	METHOD	Q-ERROR				P-ERROR			
		50%	90%	95%	99%	50%	90%	95%	99%
IMDB	PG	1.95	19.04	49.44	879.64	1.00	1.73	2.45	13.44
	MSCN*	3.24	28.54	104.20	411.90	1.07	2.31	4.00	15.66
	NeuroCard*	1.66	7.80	14.25	22.22	1.00	1.78	2.32	4.05
	PRICE	1.78	8.40	15.45	70.89	1.00	1.16	1.29	1.65
	QSPN	2.38	9.26	23.60	32.52	1.0	2.68	6.97	23.08
	Simple PRICE	1.65	5.32	8.48	47.27	1.0	1.19	1.49	1.72
	Baseball PRICE	1.91	11.01	17.09	81.21	1.0	1.65	1.80	2.81
	Specific PRICE	1.85	6.12	10.88	67.01	-	-	-	-
	Simulated PRICE	1.87	11.23	16.82	109.60	-	-	-	-
	Baseball S PRICE	1.51	6.08	12.09	36.84	-	-	-	-
STATS	PG	1.87	20.71	73.35	1600.22	1.04	2.44	4.16	20.57
	MSCN*	6.19	363.71	1609.45	9.15×10^4	1.36	5.98	13.41	59.14
	NeuroCard*	2.12	48.40	228.41	1.32×10^4	1.03	1.97	2.92	8.22
	PRICE	1.87	12.46	35.55	579.67	1.00	1.72	2.56	7.84
	Simple PRICE	2.13	14.08	41.86	1923.88	1.0	2.04	19.10	97.429
	Baseball PRICE	3.09	39.12	121.52	2548.09	1.0	26.06	65.56	98.34
	Specific PRICE	2.41	15.16	40.03	479.58	-	-	-	-
	Simulated PRICE	2.26	19.56	63.76	1142.59	-	-	-	-
	Baseball S PRICE	2.55	22.52	67.21	1311.09	-	-	-	-
ErgastF1	PG	1.60	11.30	27.47	114.24	1.00	1.64	1.98	6.39
	MSCN*	10.20	353.02	999.52	1.32×10^4	1.52	4.92	8.61	13.72
	PRICE	1.43	6.44	14.31	67.97	1.0	1.16	1.29	1.65
	Simple PRICE	1.59	7.31	13.80	37.82	1.0	1.01	1.05	1.18
	Baseball PRICE	2.10	9.74	15.46	45.90	1.0	1.00	1.02	1.11
	Specific PRICE	3.33	29.33	74.88	233.27	-	-	-	-
	Simulated PRICE	1.61	8.21	19.97	103.14	-	-	-	-
	Baseball S PRICE	2.12	10.43	16.52	43.18	-	-	-	-
Genome	PG	1.17	15.12	383.09	1.17×10^4	1.04	1.44	2.26	4.72
	MSCN*	2.84	18.91	47.45	108.48	2.68	4.38	4.39	4.40
	NeuroCard*	1.04	14.35	26.75	53.75	1.00	1.21	1.28	2.73
	PRICE	1.65	5.17	15.67	114.42	1.0	1.01	1.42	2.64
	Simple PRICE	2.17	7.50	30.04	143.11	1.0	1.24	2.28	7.93
	Baseball PRICE	2.05	11.10	14.61	49.71	1.0	2.27	4.35	10.34
	Specific PRICE	3.04	12.90	22.81	54.50	-	-	-	-
	Simulated PRICE	1.67	65.15	262.3981	8250.86	-	-	-	-
	Baseball S PRICE	1.54	6.43	21.52	141.04	-	-	-	-

'airline', 'baseball', 'basketball', and 'carcinogenesis'. We also report the performance of the model training on these databases in Table 2, labeled as "Specific PRICE". Additionally, we trained a simulated PRICE model using only the 'baseball' database, with the corresponding results shown in Table 2 under the label "Baseball S PRICE". Based on observations of these results, we draw the following conclusions:

(1) Compared to the original PRICE, although Simulated PRICE exhibits relatively weaker overall performance, its results on the first three unseen databases demonstrate that it still retains certain zero-shot cardinality estimation capabilities.

(2) Compared to the original PRICE, Specific PRICE achieves lower Q-errors on three unseen databases, IMDB, STATS, and Genome, with Q-error at the 99th percentile reduced to only 95%, 83%, and 47% of the original PRICE, respectively. Although a performance drop is observed on ErgastF1, it remains significantly more accurate than MSCN. This suggests that a model trained on only these five specific databases can achieve performance comparable to that of the model trained on all 26 databases.

(3) The comparison between Baseball S PRICE and Simulated PRICE more clearly illustrates the impact of redundant databases on model performance. The fact that training with only the 'baseball' database can even improve performance indicates that an excessive volume of insufficiently diverse data may lead to overfitting or cause the model to converge to a solution biased away from the target test distribution. Furthermore, we observe that, among all variants of PRICE models, the results of Baseball S PRICE most closely resemble those of Simple PRICE. Although these two models differ in architecture and training data composition, they share a

key commonality: both incorporate the 'baseball' database. This comparison suggests, to some extent, that the information contained in the 'baseball' database plays a dominant role in shaping the model's capabilities.

Additionally, to evaluate the generalization of these databases across test datasets, we conducted supplementary experiments using queries from the first six databases listed in Table1, along with the four standard benchmark tests.

For these six databases, we used the 'baseball' dataset and its corresponding workload as the training dataset. We selected the 'baseball' database as a single training dataset because the training dataset sorting results from our earlier algorithm indicated that the samples corresponding to the 'baseball' database contain a higher number of instances with elevated scores, while the other databases among these specific ones were ranked lower. In Figure 3, we present the variation in scores of the top five highest score databases during the final epoch over the entire training cycle. The value of scores we designed is derived from the difference in the number of samples from each database between the top 50,000 samples with the highest excess loss and the bottom 50,000 with the lowest excess loss. A negative value indicates that, for the current model's performance, there are fewer hard-to-learn samples than easier ones. From the figure, we observe a local minimum for the 'baseball' database occurring near local maxima of several other databases' curves. This suggests a clear phase transition in model behavior, where more training samples from the 'baseball' database begin to dominate the direction of gradient updates. This finding also explains why, in previous experiments, models trained with the 'baseball' database exhibited performance closer to that trained on all databases.

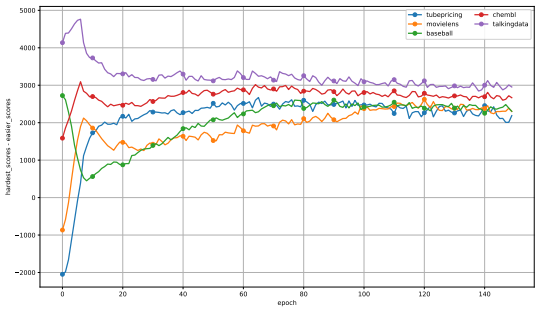

Figure 3: The evolution of scores for the databases 'tubepricing', 'movielens', 'baseball', 'chembl', 'talkingdata' over the course of training.

We selected the six databases as the test datasets: **accidents**, **carcinogenesis**, **consumer**, **hockey**, **ssb** and **talkingdata**. The reasons why we choose these databases are: (1) Average Pairwise Correlation (APC) measures the correlation between columns, which affects how easily a model fits the data distribution. Among these datasets, three have an APC higher than that of the **baseball** dataset, while the remaining three have a lower APC. (2) These databases are divided into two groups based on APC values, with each group containing chain, star, and mixed-join forms. From the corresponding training datasets $\mathcal{X}_i$ of these databases, we sampled 150 queries each to form the test dataset.

We use approximately 1×10^6 queries collected from the 'baseball' database as the training dataset to obtain the pretrained CardEst method with one dataset, denoted as "Baseball PRICE". We compare the Q-error and End-to-end Time of Baseball PRICE with other baselines. Other baselines use the same parameter settings as before.

Fig. 4 shows the evaluation Q-error (top) and End-to-end Time (bottom) of the cardinality estimators. Here, we do not display the values for PostgreSQL at the 90th and 95th percentile Q-error because they exceeded 1000 across all test datasets. In these test workloads, Baseball PRICE generalizes well and is statistically efficient. These datasets exhibit diverse join schemas, limiting NeuroCard and QSPN to only a few compatible datasets. In contrast, Baseball PRICE achieves consistently acceptable Q-errors and End-to-end Times across all test datasets. Compared to PostgreSQL, Baseball PRICE achieves a lower Q-error across all test sets. In End-to-end times, it demonstrates some performance improvements over MSCN in 3 databases, with 1.03×, 1.07× and 1.03× faster compared to using MSCN cardinalities in **carcinogenesis**, **hockey** and **talkingdata**. Regarding Q-error, although Baseball PRICE outperforms MSCN on only two databases, it is important to note that Baseball PRICE was trained without any information from these test databases. Despite this zero-shot setting, it still manages to achieve a 95th percentile Q-error below 100 on five of the databases. For such zero-shot models, performance can be further enhanced by employing a few-shot fine-tuning.

We also evaluate Baseball PRICE on the same four previously used unseen new datasets. As shown in Table 2, it shows comparable or even superior performance across all scenarios to PRICE, except for the stats dataset. We attribute its suboptimal performance on the stats dataset to the fact that the low-level attribute information provided by the baseball dataset may be insufficient to handle the complexity of stats dataset.

5.4 Advantages and Characteristics of the Specific Databases

For learning-based cardinality estimation methods, the model training time scales linearly with the size of the training data. Both Simple PRICE and Baseball PRICE require fewer training resources compared to the original PRICE, significantly reducing the complexity of the training. As evidenced in Fig. 5, the remixed dataset and single-dataset strategies substantially reduce training time. Baseball Price, while closely matching PRICE's performance, trains in just 6% of PRICE's training time. The cold-start problem remains a critical weakness for query-driven cardinality estimators, as constructing a training dataset demands substantial overhead, which need to execute thousands of SQL queries to obtain true cardinalities. While exiting pretrained CardEst methods often aggravate this problem because of requiring so many queries from multiple datasets, Baseball PRICE demonstrates that a single well-chosen dataset (for example, baseball) may suffice to achieve competitive performance.

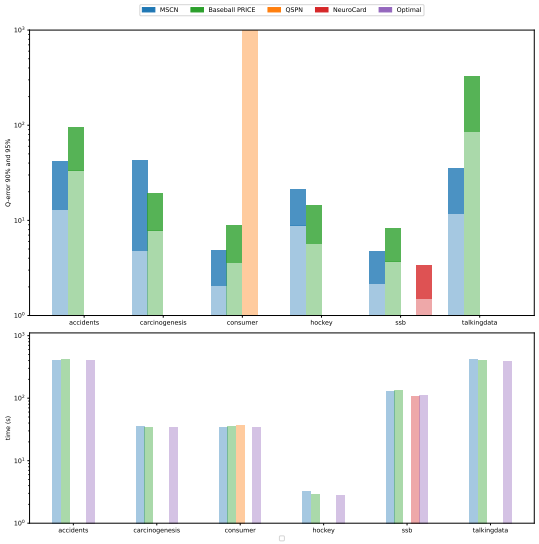

Figure 4: Evaluation Q-error (top) and End-to-end Time (bottom) of Cardinality Estimation Models.

Figure 6 displays the proportion of learnt samples within each specific type of sample in the 'baseball' database after the completion of training. We categorize all queries based on having the same number of tables, join keys, join columns, and predicate columns, as their corresponding tensors share identical lengths prior to the embedding layer. In the figure, the blue and red bars represent the number of samples with zero loss and the total number of samples for each type, respectively, while the line graph illustrates the exact proportions. The analysis reveals that for the dominant query types in the baseball database, specifically those with a total sample count exceeding 1,000 and the proportion of samples fully learnt by the model falls within the range of 20% to 40%.

6 Related Work

Our work primarily addresses research questions related to learning-based cardinality estimators. Learning-based cardinality estimators are typically divided into query-driven and data-driven models based on the pattern of inputs required during training. Query-driven methods[7, 15, 34] aim to establish a regression model between the data ranges abstracted from queries and the true cardinalities, indirectly fitting the data distribution under the schema. Data-driven methods[13, 25, 27, 28, 30**?** , 31] directly model the data distribution in the schema using the principles of probabilistic graphical models in machine learning. The learning objectives of these two types of methods are not contradictory; hence, some research combines these two approaches into hybrid-driven methods. Some hybrid methods [7, 17, 26] use the loss obtained from queries and true cardinalities as other penalty items for the learning objective to improve the performance of data-driven methods and introduce several strengths of query-driven methods.

The methodological approach employed in this work is primarily inspired by techniques in data reweighting and data pruning. Distributionally Robust Optimization (DRO) methods are typically used in the deep learning field[20, 22] to enhance the generalization ability of models to obtain robust models by defining an uncertainty set first. Group DRO[22], emerging on this basis, reduces training difficulty at the cost of fewer degrees of freedom. Our approach is inspired by DoReMi[29], it uses a small-scale proxy model to optimize the data mixing for training large language models more efficiently. We use group DRO to calculate the contribution of each training dataset to the model's performance. Generally, the effectiveness of the DoReMi method depends on the performance of its proxy model. While alternative approaches such as DoGE[9] have been proposed, for example, using gradient dynamics during training to optimize domain-specific weights, we observed that the model provided by PRICE exhibits more stable performance than our own trained versions. Therefore, we ultimately adopted the DoReMi method as our foundation. On the other hand, methods such as GraNd and EL2N scores[21] can prune significant portions of the training data without sacrificing test accuracy. These approaches compute sample-level scores based on loss dynamics during training in classification tasks, and training on the highest-scoring samples has been shown to perform on par with or even outperform training on the full dataset. In contrast, our method aims to identify which database-specific training subsets contain more of these highest-scoring examples. Moreover, we compute these scores using variations in excess loss rather than conventional training loss.

7 Conclusion and Future Work

In this article, we begin by analyzing one of the key challenges facing existing pretrained cardinality estimators: significant disparities in the importance of different training datasets. To address this, we propose a group distributionally robust optimization (group DRO) based method to estimate the contribution of each domain within the training dataset. Based on the PRICE model's training dataset, we develop some simplified version that significantly reduces the scale of the pretraining dataset while having nearly performance. We propose a new pretrained zero-shot model and train it on the simplified datasets. Experimental results demonstrate that these simplified training datasets are not only effective for the PRICE model but also exhibit strong generalizability. We evaluated models trained on specific databases across multiple additional test workloads, with performance metrics in both Q-error and end-to-end execution time confirming that the approach generalizes well and maintains statistical efficiency.

In future work, we plan to develop more efficient methods for identifying valuable subsets within pretraining datasets, provide theoretical guarantees for our current approach, and design an improved pretrained cardinality estimator.

8 Artifacts

The code used in this article is open source at https://github.com/spiceandwolf/PRICE.git.

References

[1] [n. d.]. ErgastF1. https://relational-data.org/dataset/ErgastF1. [Accessed 26-11-2024].
[2] [n. d.]. Genome. https://relational-data.org/dataset/VisualGenome. [Accessed 26-11-2024].

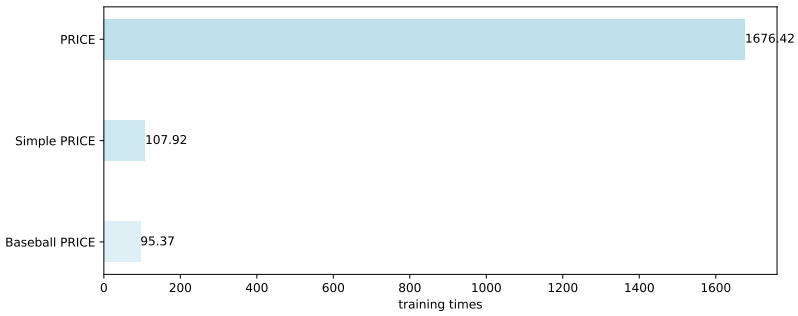

Figure 5: Training Times of PRICE, Simple PRICE and Baseball PRICE.

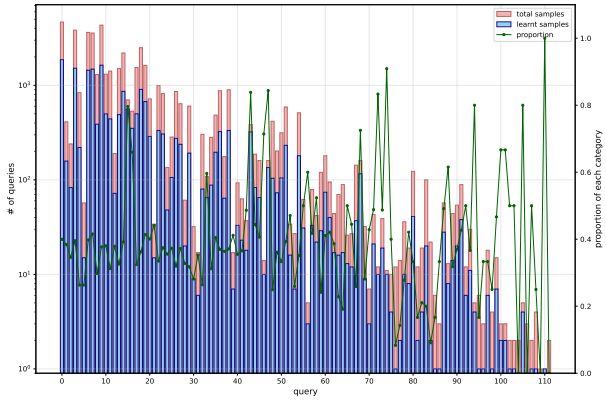

Figure 6: Sample Counts and Proportion of Learnt Samples by Query Category in the Baseball Database. The left y-axis, using a logarithmic scale, indicates the '# of queries,' representing both the number of learnt samples and the total number of samples. The right y-axis indicates the 'proportion of each category,' showing the ratio of learnt samples to the total number of samples for each query category. Each item on the x-axis corresponds to a distinct query set.

[3] [n. d.]. IMDB. http://homepages.cwi.nl/~boncz/job/imdb.tgz. [Accessed 26-11-2024].

[4] [n. d.]. STATS. https://relational.fit.cvut.cz/dataset/Stats. [Accessed 26-11-2024].

[5] 1996. PostgreSQL Global Development Group. https://www.postgresql.org. [Accessed 26-11-2024].

[6] Yannis Chronis, Yawen Wang, Yu Gan, Sami Abu-El-Haija, Chelsea Lin, Carsten Binnig, and Fatma Özcan. 2024. CardBench: A Benchmark for Learned Cardinality Estimation in Relational Databases. *arXiv preprint arXiv:2408.16170* (2024).

[7] Anshuman Dutt, Chi Wang, Azade Nazi, Srikanth Kandula, Vivek Narasayya, and Surajit Chaudhuri. 2019. Selectivity estimation for range predicates using lightweight models. *Proceedings of the VLDB Endowment* 12, 9 (2019), 1044–1057.

[8] Logan Engstrom, Axel Feldmann, and Aleksander Madry. 2024. DsDm: Model-Aware Dataset Selection with Datamodels. arXiv:2401.12926 [cs.LG] https://arxiv.org/abs/2401.12926

[9] Simin Fan, Matteo Pagliardini, and Martin Jaggi. 2024. DoGE: Domain Reweighting with Generalization Estimation. arXiv:2310.15393 [cs.LG] https://arxiv.org/abs/2310.15393

[10] Yuxing Han, Ziniu Wu, Peizhi Wu, Rong Zhu, Jingyi Yang, Liang Wei Tan, Kai Zeng, Gao Cong, Yanzhao Qin, Andreas Pfadler, et al. 2021. Cardinality estimation in dbms: A comprehensive benchmark evaluation. *arXiv preprint arXiv:2109.05877* (2021).

[11] Nan He, Weichen Xiong, Hanwen Liu, Yi Liao, Lei Ding, Kai Zhang, Guohua Tang, Xiao Han, and Wei Yang. 2024. SoftDedup: an Efficient Data Reweighting Method for Speeding Up Language Model Pre-training. arXiv:2407.06654 [cs.CL] https://arxiv.org/abs/2407.06654

[12] Benjamin Hilprecht and Carsten Binnig. 2022. Zero-shot cost models for out-of-the-box learned cost prediction. *arXiv preprint arXiv:2201.00561* (2022).

[13] Benjamin Hilprecht, Andreas Schmidt, Moritz Kulessa, Alejandro Molina, Kristian Kersting, and Carsten Binnig. 2019. Deepdb: Learn from data, not from queries! *arXiv preprint arXiv:1909.00607* (2019).

[14] Diederik P. Kingma and Jimmy Ba. 2017. Adam: A Method for Stochastic Optimization. arXiv:1412.6980 [cs.LG] https://arxiv.org/abs/1412.6980

[15] Andreas Kipf, Thomas Kipf, Bernhard Radke, Viktor Leis, Peter Boncz, and Alfons Kemper. 2018. Learned cardinalities: Estimating correlated joins with deep learning. *arXiv preprint arXiv:1809.00677* (2018).

[16] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2015. How good are query optimizers, really? *Proceedings of the VLDB Endowment* 9, 3 (2015), 204–215.

[17] Jiawei Liu, Ju Fan, Tongyu Liu, Kai Zeng, Jiannan Wang, Quehuan Liu, Tao Ye, and Nan Tang. 2025. A Unified Model for Cardinality Estimation by Learning from Data and Queries via Sum-Product Networks. arXiv:2505.08318 [cs.DB] https://arxiv.org/abs/2505.08318

[18] Yao Lu, Srikanth Kandula, Arnd Christian König, and Surajit Chaudhuri. 2021. Pre-training summarization models of structured datasets for cardinality estimation. *Proc. VLDB Endow.* 15, 3 (Nov. 2021), 414–426. doi:10.14778/3494124.3494127

[19] Sören Mindermann, Jan M Brauner, Muhammed T Razzak, Mrinank Sharma, Andreas Kirsch, Winnie Xu, Benedikt Höltgen, Aidan N Gomez, Adrien Morisot, Sebastian Farquhar, et al. 2022. Prioritized training on points that are learnable, worth learning, and not yet learnt. In *International Conference on Machine Learning*. PMLR, 15630–15649.

[20] Yonatan Oren, Shiori Sagawa, Tatsunori B Hashimoto, and Percy Liang. 2019. Distributionally robust language modeling. *arXiv preprint arXiv:1909.02060* (2019).

[21] Mansheej Paul, Surya Ganguli, and Gintare Karolina Dziugaite. 2021. Deep learning on a data diet: Finding important examples early in training. *Advances in neural information processing systems* 34 (2021), 20596–20607.

[22] Shiori Sagawa, Pang Wei Koh, Tatsunori B Hashimoto, and Percy Liang. 2019. Distributionally robust neural networks for group shifts: On the importance of regularization for worst-case generalization. *arXiv preprint arXiv:1911.08731* (2019).

[23] Ben Sorscher, Robert Geirhos, Shashank Shekhar, Surya Ganguli, and Ari Morcos. 2022. Beyond neural scaling laws: beating power law scaling via data pruning. *Advances in Neural Information Processing Systems* 35 (2022), 19523–19536.

[24] Xiu Tang, Sai Wu, Mingli Song, Shanshan Ying, Feifei Li, and Gang Chen. 2022. PreQR: pre-training representation for SQL understanding. In *Proceedings of the 2022 international conference on management of data*. 204–216.

[25] Jiayi Wang, Chengliang Chai, Jiabin Liu, and Guoliang Li. 2021. FACE: A normalizing flow based cardinality estimator. *Proceedings of the VLDB Endowment* 15, 1 (2021), 72–84.

[26] Peizhi Wu and Gao Cong. 2021. A unified deep model of learning from both data and queries for cardinality estimation. In *Proceedings of the 2021 International Conference on Management of Data*. 2009–2022.

[27] Ziniu Wu, Parimarjan Negi, Mohammad Alizadeh, Tim Kraska, and Samuel Madden. 2023. FactorJoin: a new cardinality estimation framework for join queries. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–27.

[28] Ziniu Wu and Amir Shaikhha. 2012. BayesCard: A Unified Bayesian Framework for Cardinality Estimation. CoRR abs/2012.14743 (2020).

[29] Sang Michael Xie, Hieu Pham, Xuanyi Dong, Nan Du, Hanxiao Liu, Yifeng Lu, Percy S Liang, Quoc V Le, Tengyu Ma, and Adams Wei Yu. 2024. Doremi: Optimizing data mixtures speeds up language model pretraining. *Advances in Neural Information Processing Systems* 36 (2024).

[30] Zongheng Yang, Amog Kamsetty, Sifei Luan, Eric Liang, Yan Duan, Xi Chen, and Ion Stoica. 2020. NeuroCard: one cardinality estimator for all tables. *arXiv*

preprint arXiv:2006.08109 (2020).

[31] Zongheng Yang, Eric Liang, Amog Kamsetty, Chenggang Wu, Yan Duan, Xi Chen, Pieter Abbeel, Joseph M Hellerstein, Sanjay Krishnan, and Ion Stoica. 2019. Deep unsupervised cardinality estimation. *arXiv preprint arXiv:1905.04278* (2019).

[32] Tianjing Zeng, Junwei Lan, Jiahong Ma, Wenqing Wei, Rong Zhu, Pengfei Li, Bolin Ding, Defu Lian, Zhewei Wei, and Jingren Zhou. 2024. PRICE: A Pretrained Model for Cross-Database Cardinality Estimation. *arXiv preprint arXiv:2406.01027* (2024).

[33] Kaixin Zhang, Hongzhi Wang, Yabin Lu, Ziqi Li, Chang Shu, Yu Yan, and Donghua Yang. 2024. Duet: efficient and scalable hybriD neUral rElation undersTanding. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 56–69.

[34] Kangfei Zhao, Jeffrey Xu Yu, Zongyan He, Rui Li, and Hao Zhang. 2022. Lightweight and accurate cardinality estimation by neural network gaussian process. In *Proceedings of the 2022 International Conference on Management of Data*. 973–987.

[35] Rong Zhu, Ziniu Wu, Yuxing Han, Kai Zeng, Andreas Pfadler, Zhengping Qian, Jingren Zhou, and Bin Cui. 2021. FLAT: Fast, Lightweight and Accurate Method for Cardinality Estimation. arXiv:2011.09022 [cs.DB] https://arxiv.org/abs/2011.09022

A Domain Weights from Price

Table 3: Original Weights and Our Reweights of the 26 Training Datasets in PRICE[32]. Baseline training dataset weights are calculated based on the number of SQL queries in the corresponding workload. Simplified represents the reweights of training dataset.

Training Sets	Reweight
accidents	0.0023
airline	0.0140
baseball	0.1651
basketball	0.0178
carcinogenesis	0.0015
ccs	0.00002
chembl	0.1306
consumer	0.00001
credit	0.0001
employee	0.0051
financial	0.0007
fnhk	0.0531
grants	0.0117
hepatitis	0.00002
hockey	0.0163
legalacts	0.00002
movielens	0.0683
sakila	0.00001
sap	0.00007
seznam	0.0270
ssb	0.00001
talkingdata	0.4418
telstra	0.0033
tournament	0.020
tpc_h	0.0020
tubepricing	0.0180